



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Московский государственный технический университет имени

Н.Э. Баумана

(национальный исследовательский университет)»

(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5 ПО ДИСЦИПЛИНЕ: ТИПЫ И СТРУКТУРЫ ДАННЫХ

«Обработка деревьев»

Студент: Колоколов Г.И.

Группа: ИУ7-35Б

Название предприятия **НУК ИУ МГТУ им. Н. Э. Баумана**

Студент _____ Колоколов Г. И.

Преподаватель _____ Никульшина Т. А.

Оглавление

Цель работы.....	3
Постановка задачи	3
Техническое задание	3
Входные данные.....	3
Выходные данные	3
Задача программы.....	4
Описание структур данных	5
Описание алгоритма программы.....	6
Описание алгоритма добавления элемента в дерево.....	6
Описание алгоритма удаления элемента из дерева	6
Сравнение используемого времени работы алгоритма.....	6
Сравнение деревьев	7
Выводы по проделанной работе	7
Контрольные вопросы	8

Цель работы

Получить навыки применения двоичных деревьев, реализовать основные операции над деревьями: обход деревьев, включение, исключение и поиск узлов.

Постановка задачи

Построить дерево в соответствии со своим вариантом задания. Вывести его на экран в виде дерева. Реализовать основные операции работы с деревом: обход дерева, включение, исключение и поиск узлов. Сравнить эффективность алгоритмов сортировки и поиска в зависимости от высоты деревьев и степени их ветвления.

Техническое задание

Каждый узел дерева содержит информацию о студенте: Фамилия, средний балл. Сформировать дерево бинарного поиска по фамилии. Написать программу обхода дерева и удаления всех студентов со средним баллом ниже или равно 2.0. Вывести исходное дерево и дерево после удаления. Сравнить время удаления в дереве, построенном по фамилии со временем перестроения по баллу и удаления в нем.

Входные данные

1. Целое число (от 0 до 7 включительно) - индекс команды, которая будет исполнена программой (см. раздел Реализованные возможности программы)
2. Строка - фамилия студента при поиске или удалении в дереве
3. Вещественное число - средний балл студента при его добавлении

Выходные данные

1. Картинка - визуализация дерева
2. Данные о студенте при его поиске в дереве

3. Результаты сравнения памяти и времени используемых алгоритмов и структур данных
4. Ошибка при работе с программой

Задача программы

1. Вывод назначения и ограничений программы
2. Обработка и валидация ввода
3. Ввод данных для работы программы
4. Если введенные данные корректны, то выполнить запрошенное действие, иначе вывести сообщение об ошибке

Реализованные возможности программы

1. Добавить элемент в дерево
2. Найти элемент в дереве
3. Удалить элемент из дерева
4. Вывести значения всех узлов дерева
5. Вывести картинку-визуализацию графа дерева
6. Удалить все узлы с средним баллом ниже 2
7. Получить сравнение типов данных и алгоритмов
8. Выйти из программы

Код завершения программы

Код завершения	Символьное обозначение	Описание кода
SUCCESS	0	Успешный выход из программы
NULLPTR_ERROR	1	В функцию был передан некорректный указатель

ALLOC_ERROR	2	Ошибка выделения памяти под стек/узел стека
NAN_ERROR	3	Ошибка перевода строки в число
EMPTY_GRAPH	4	Пустое дерево

Описание структур данных

```
typedef struct TreeNode TreeNode;

// структура, описывающая один узел в дереве
struct TreeNode
{
    StudentData *data;

    TreeNode *left;
    TreeNode *right;
};

// вставка нового элемента в дерева
TreeNode *treeInsert(TreeNode *head, StudentData *data, int (*comparator)(const StudentData *data,
const StudentData *), int *rc);

// удаление узла студента по фамилии из дерева
TreeNode *treeRemove(TreeNode *head, const char *name);

// поиск узла по фамилии студента в дереве
TreeNode *treeFind(TreeNode *head, const char *prefix);

// фильтрация узлов дерева (удаление по условию)
TreeNode *filterTree(TreeNode *head, bool (*filter)(const StudentData *node));

// удаление и очистка дерева
void destroyTree(TreeNode **node);
```

```
// обход в глубину для дерева
void depthFirstSearch(TreeNode *head, void (*action)(TreeNode *, void *), void *param);

// освобождение памяти одного узла
void freeNode(TreeNode *node);
```

Описание алгоритма программы

1. Программа выводит правила и ограничения
2. Программа выводит меню для пользователя
3. Программа будет продолжать предлагать пользователю команды до тех пор, пока он выберет пункт завершения программы

Описание алгоритма добавления элемента в дерево

Если узла не существует, то вернуть новый узел с заданными параметрами. Если узел существует, то сравнить данные для добавления с данными узла. Если результат сравнения окажется меньше, то рекурсивно перейти к левому потомку, если больше, то к правому. Если данные равны, то элемент уже существует в дереве и его не надо добавлять.

Описание алгоритма удаления элемента из дерева

Для удаления узла с указанным ключом сначала происходит его поиск. В случае, если узел найден, то он удаляется. Если удаляемый узел является концевым, то просто удаляется ссылка на него. Если удаляемый узел имеет одного потомка, в этом случае переадресуется ссылка на этого потомка. Если удаляемый узел имеет двух потомков, то на его место ставится самый правый потомок из левого поддерева или самый левый потомок из правого поддерева.

Сравнение используемого времени работы алгоритма

Замеры времени происходят путем запуска функции вставки и удаления элемента несколько раз (50 в моей программе).

Сравнение деревьев

Размеры дерева

Элементы	Размер
500	12000
1000	24000
5000	120000

Вставка в дерево (бамбук). В таком случае дерево вырождается в список

Элементы	Время вставки	Время удаления
500	2504	1175
1000	5186	2528
5000	29280	13387

Вставка и удаление в отсортированном дереве

Элементы	Время вставки	Время удаления
500	92	150
1000	104	121
5000	124	159

Сравнения время фильтрации дерева построенного по фамилии студента, с деревом, построенным на среднем балле

Элементы	Фамилия	Средний балл
500	7498	6474
1000	13621	12990
5000	67641	66502

Выводы по проделанной работе

Эффективность бинарного дерева поиска зависит от того, в каком порядке оно было сформировано. Бинарное дерево поиска может выродиться в линию, что ухудшит асимптотику поиска в дереве до $O(n)$, в среднем поиск работает за $O(\log n)$. Таким образом выгодно использовать бинарное дерево тогда, когда нам часто надо искать какие-то данные.

Контрольные вопросы

1. Что такое дерево?

Дерево - это нелинейная структура данных, используемая для представления иерархических связей, имеющих отношение «один ко многим». Так же это связный ациклический граф.

Память под дерево выделяется отдельно под каждый узел дерева, таким образом суммарная память будет равна: количество узлов * размер одного узла.

2. Какие бывают типы деревьев?

Существует очень много различных древовидных структур, таких как: двоичное дерево поиска, самобалансирующееся дерево, дерево отрезков, декартово дерево и т.д.

3. Какие стандартные операции возможны над деревьями?

- Вставка элемента
- Удаление элемента
- Поиск элемента
- Обход дерева

4. Что такое дерево двоичного поиска?

Двоичным деревом поиска называется такое дерево, у которого каждый узел имеет не более двух потомков